

Integración en arquitectura do software

Reutilización

- Sistemas **COTS**
 - *Commercial Off-The-Shelf*
 - Compoñentes ou servizos de propósito xeral proporcionados por un terceiro
 - Aplicable a hardware e a software
 - Coste reducido
- Sistemas **legacy**
 - Sistemas herdados
 - Aplicable a hardware e a software
 - Amortización de custo

Reutilización por integración

Busca conseguir que aplicacións, servizos e/ou **compoñentes dispares traballen xuntos** para proporcionar un conxunto de **funcionalidades de maneira unificada**.

Características dun escenario de integración

- Combinación de elementos propios (*legacy*) e adquiridos a terceiros (*COTS*)
- Execución en diferentes máquinas físicas, plataformas diferentes, distintas ubicacións
- Non necesariamente deseñados para ser integrados, pouco flexibles.

Malia certas características xerais, as necesidades de integración varían en cada **escenario concreto** de integración

Estilos de integración

Transferencia de ficheiros

Os compoñentes xeran ficheiros coa información a compartir que son consumidos por outros compoñentes, e viceversa.

- Convención de nomeado e ubicación
- Política de unicidade de nomes
- Responsabilidade de borrado
- Mecanismo de *locking* ou *sincronización*
- Custe de procesado dos ficheiros

- Protección de acceso
- Se precisa que os datos estean dispoñibles rapidamente e é posible acordar un formato único: *Compartición de fontes de datos*
- Se se precisa integrar funcionalidade máis que datos: *Invocación de procedementos remotos*
- Se o intercambio de datos é pequeno e ten como obxectivo solicitar a execución de servizos: *Paso de mensaxes*.

Compartición de fontes de datos

Os compoñentes usan un repositorio de datos onde almacenan a información que queren compartir

- Esquema de datos común
- Punto crítico de rendemento
- Distribución da fonte de datos (sincronización, rendemento)
- Protección do acceso (usuarios, vistas)
- Se se precisa integrar funcionalidade máis que datos: *Invocación de procedementos remotos*
- Se o intercambio de datos é pequeno e interesa máis o tipo de dato en cada intercambio que utiliza un resquema global: *Paso de mensaxes*

Invocación de procedementos remotos

Cada compoñente pon a disposición do resto unha API de intercambio de datos/acceso a funcionalidades que pode invocarse de xeito remoto

- Cada compoñente pode ofrecer unha API diferente a distintos interlocutores
- Risco de tratar as invocacións remotas como locais (latencia, dispoñibilidade)
- Risco de *secuenciación* (imposición da orde nas chamadas)
- Mecanismos de seguridade a nivel de compoñente
- Para integrar compoñentes de xeito desacoplado e asíncrono: *Paso de mensaxes*

Paso de mensaxes

Cada compoñente conéctase a unha canle de intercambio de mensaxes común, onde se intercambian datos/peticións de servizos

- A secuenciación evítase reordeando as mensaxes á hora de procesalas

- Probas e depuración son máis complexos
- Adaptadores máis complexos (máis que "tradutores de tecnoloxía", como na invocación de procedementos remotos)
- Definición de mensaxes, envío e recepción, tradución, canal, enrutados